



evaluation & edit distance

LING 430 Computational Linguistics Fall 2026

lecture 5b

Sep 24 2026

Siyu Liang

siyu.liang@rice.edu

last time

- morphology
 - morphemes, morphemes per word, morphological typology
- normalization
 - case, accents, stems, lemmas

agenda

- evaluation
- edit distance



today's notebook

class05b.ipynb

- course website → week 5 → activity
- **File → Save a copy in Drive**



evaluation

evaluation

- all systems need evaluation, but what with?
- a **gold standard**: a set of inputs labeled with the correct answer by a person
- the system answers the same inputs
 - we then compare between the system's output and the gold standard output

evaluation

- what is the gold standard?
 - usually, *someone* labeled every item
 - an annotator, with guidelines, disagreeing with another annotator and adjudicating (inter-annotator agreement)
 - the gold standard encodes somebody's judgments, and a score inherits every one of them

evaluation

accuracy

- **accuracy**: the proportion of correct predictions out of all answers

```
gold = [1, 1, 0, 1, 0, 0, 1, 1, 0, 1]
```

```
pred = [1, 1, 1, 0, 0, 0, 1, 0, 0, 1]
```

```
correct = 0
```

```
for g, p in zip(gold, pred):
```

```
    if g == p:
```

```
        correct = correct + 1
```

```
correct / len(gold)
```

```
→ 0.7
```

evaluation

the accuracy paradox

- 1,000 patients, one of them sick
 - system A says healthy to everyone: accuracy $999 / 1,000 = 99.9\%$
 - system B finds the sick patient and flags nine healthy ones: accuracy $991 / 1,000 = 99.1\%$
- which one do you want in a hospital?

truth	A says healthy	A says sick	B says healthy	B says sick
healthy (999)	999	0	990	9
sick (1)	1	0	0	1

evaluation

confusion matrix

- we need more for evaluation other than accuracy alone
- results can be organized into a **confusion matrix**

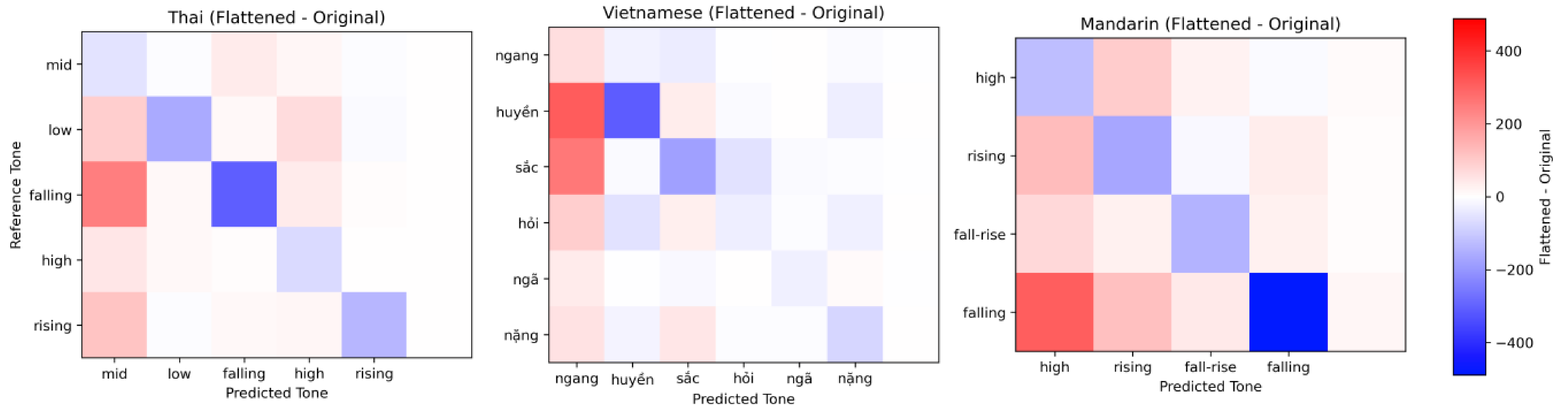
		Predicted condition	
		Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

Total population
 $= P + N$

evaluation

confusion matrix

- what if you have more than two answers?
 - you can overlay a **heatmap**

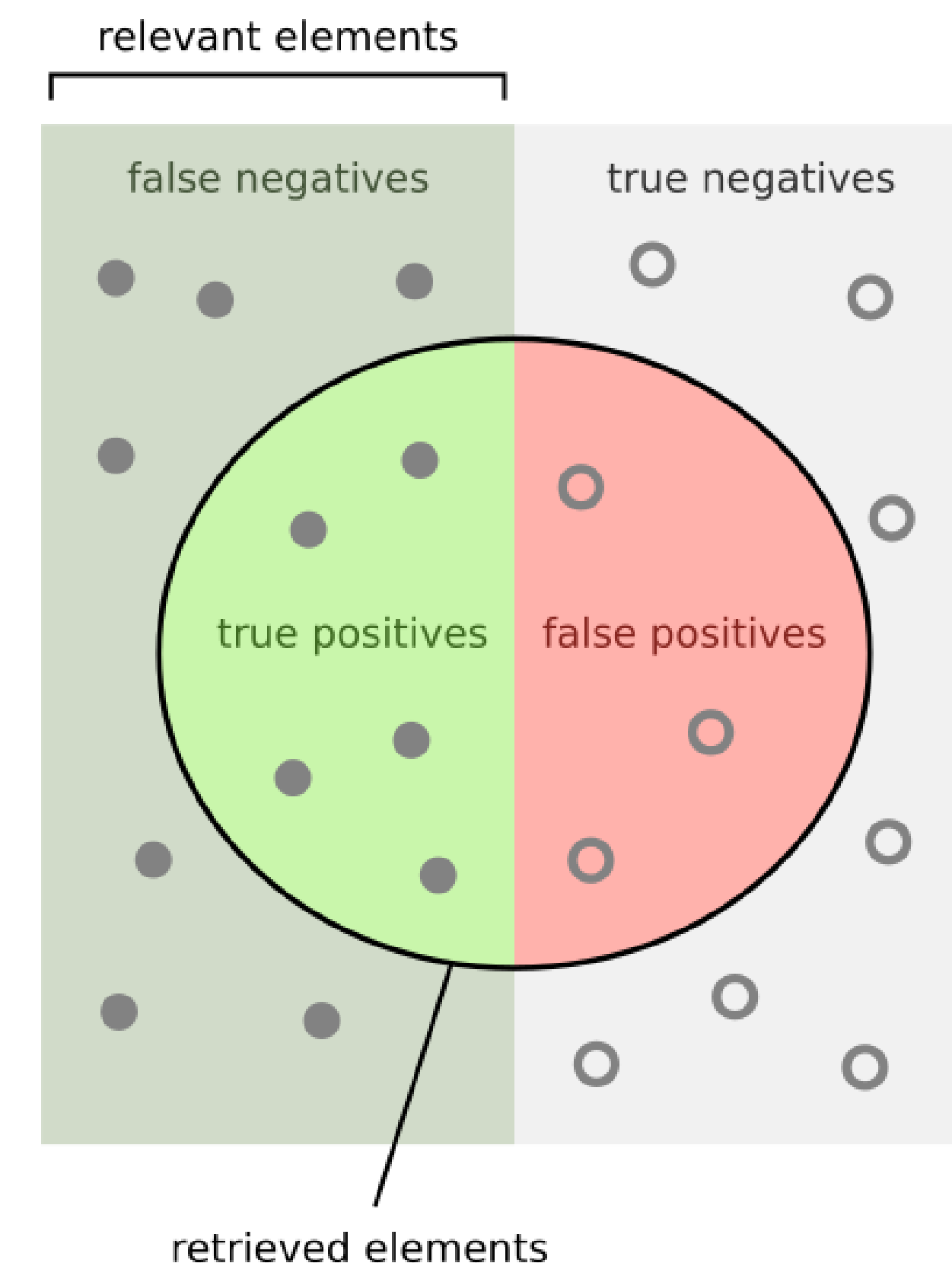


evaluation

precision & recall

- with a confusion matrix, you can calculate **precision** and **recall**

	formula	in words
precision	$TP / (TP + FP)$	of what the system said yes to, how much was right?
recall	$TP / (TP + FN)$	of what was really there, how much did it find?



How many retrieved items are relevant?

Precision = $\frac{\text{true positives}}{\text{true positives} + \text{false positives}}$

How many relevant items are retrieved?

Recall = $\frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$

evaluation

- there are two ways to cheat
 - 1. **yes to everything**
 - recall 100%, and precision equals the rate of positives, 1 in 1,000 here
 - 2. **no to everything**
 - precision undefined, recall 0%
- report both numbers to help prevent it
- before choosing any system, ask: what the dumbest strategy scores on the same data?
 - sometimes a model just performs **at chance**
 - guessing a coin toss at 0.5 accuracy doesn't mean you have superpower

evaluation

F1

- **F1 score** is the harmonic mean of precision and recall
 - $F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
- the harmonic mean balances each calculation, so no cheat can buy a good F1 with one inflated metric
- F1 ignores true negatives too, so it is a score about the yes class only



not this F1

evaluation

scikit-learn



- **scikit-learn** is a major Python library for machine-learning
 - build, train, and evaluate statistical & machine learning models

```
from sklearn.metrics import accuracy_score, precision_score,  
recall_score, f1_score
```

```
accuracy_score(gold, pred), precision_score(gold, pred)
```

```
→ (0.7, 0.8)
```

```
recall_score(gold, pred), f1_score(gold, pred)
```

```
→ (0.6666666666666666, 0.7272727272727272)
```

activity 1

- you're evaluating an algorithm to detect hate-speech on a social media platform
 - 1: hate-speech 🤬; 0: safe ✅
- calculate the four counts (TP, TN, FP, FN), accuracy, precision and recall
- how does each system do?

gold:	[1, 1, 0, 1, 0, 0, 1, 1, 0, 1]
A:	[1, 1, 1, 0, 0, 0, 1, 0, 0, 1]
B:	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
C:	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
D:	[1, 1, 0, 1, 0, 0, 1, 1, 0, 1]

activity 1

debrief

- the baseline is: what does the dumbest system score on this data?
 - here it is already 0.75

system	accuracy	precision	recall	F1
A	0.70	0.80	0.67	0.73
B, all no	0.40	undefined	0.00	0.00
C, all yes	0.60	0.60	1.00	0.75
D	1.00	1.00	1.00	1.00

discussion

- suppose you're building these systems
 - wake-word detection for a voice assistant (1: user said 'Hey Siri')
 - hallucination checker for LLM answers (1: hallucination)
 - AI-writing detection tool for catching cheating (1: AI-generated content)
 - dating app recommendations (1: a match you'll like)
- decide: optimize precision or recall?
 - explain in the template "a false ____ costs ____, so we chose ____"

evaluation

- a system's false positives are usually obvious
 - the wrong hits, the flagged email, the rejected form
- its false negatives are not
 - the corpus hits the pattern missed, the real word the spellchecker rejected, the post the filter let through
- measuring recall means going looking for what you missed
 - labeling a sample of everything, not just of the system's output
- which is why deployed systems drift toward precision
 - the visible errors get fixed



evaluation

- the metric can not be used to make the decision alone
 - a person will have to decide, usually on someone else's behalf
- a score says how often a system fails
 - reading and explaining the errors can help us understand *why*
- we'll talk about error analysis later in the class



edit distance

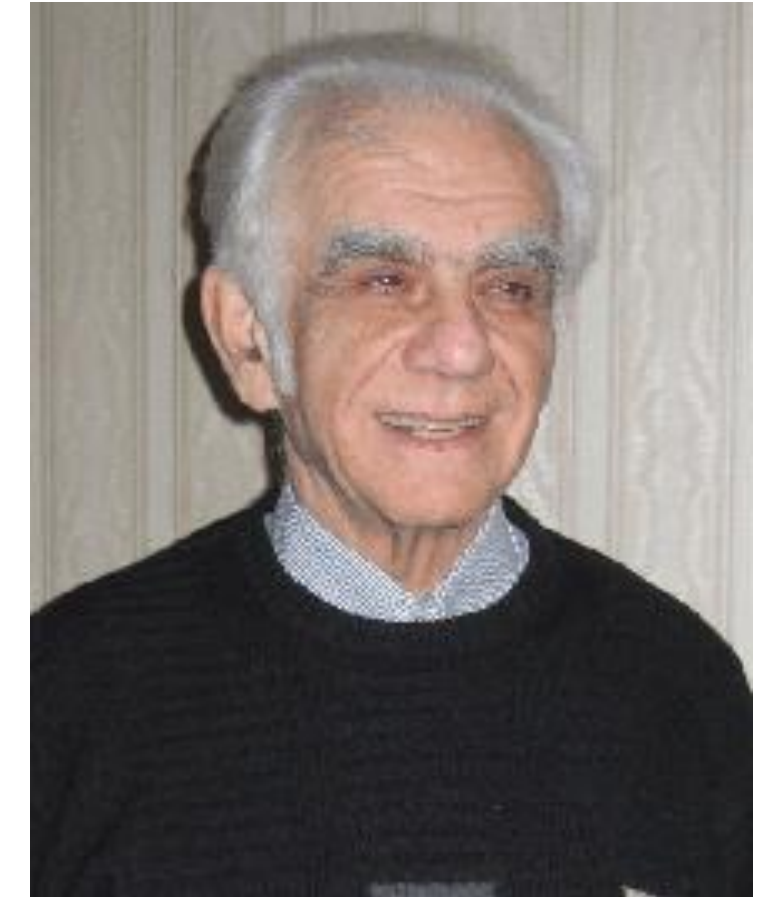
ways of evaluation

detection vs. correction

- **detection** is a lookup
 - is the word in the word list?
 - yes/no
- **correction** is a ranking
 - if someone typed *briht*
 - which real word was meant by?
 - *bright, brit, birth, brunt, brat, brought*
 - we want the *closest* one
 - but closest by *what measure*?

edit distance

- edit distance makes use of three **operations**
 - **insertion** ($AB \rightarrow ACB$)
 - **deletion** ($AB \rightarrow A$)
 - **substitution** ($A \rightarrow B$)
- the **minimum edit distance** between two strings is the smallest number of operations that turns one into the other
- **Levenshtein distance** is the most used one, where each operation costs 1
 - one version charges 2 for a substitution (a deletion plus an insertion)



Vladimir I. Levenshtein

edit distance

- *intention to execution*: delete I, substitute N $\rightarrow$ E, substitute T $\rightarrow$ X, insert C, substitute N $\rightarrow$ U; match the rest
- an **alignment** pairs each piece of one string with a piece of the other, or with nothing

I	N	T	E	*	N	T	I	O	N
*	E	X	E	C	U	T	I	O	N
d	s	s		i	s				

activity 2

- someone typed *briht*
 - maybe the intended word is *bright*, *brit*, *birth*, *brunt*, *brat*, *brought*
- first rank the six by how likely each was the intended word to your intuition
- then with a partner: count the fewest insertions, deletions and substitutions from *briht* to each, and rank by the count
- tips: write the two words one above the other and pair the letters; a substitution is one operation, not two; only the three operations, no swapping
- how does your intuition and your count compare?

activity 2

debrief

candidate	cheapest path from briht	distance
bright	insert g	1
brit	delete h	1
brunt	substitute $i \rightarrow u$, $h \rightarrow n$	2
brat	delete h, substitute $i \rightarrow a$	2
birth	three operations, however you pair them	3
brought	substitute $i \rightarrow o$, insert u, insert g	3

edit distance

in Python

- `nltk.edit_distance()` calculates the edit distance
- use the parameter `substitution_cost=...` to adjust the cost of substitution

```
nltk.edit_distance("graffe", "giraffe")
```

→ 1

```
nltk.edit_distance("teh", "the")
```

→ 2

```
nltk.edit_distance("teh", "the", transpositions=True)
```

→ 1

edit distance

- edit distance is used a lot in evaluation of NLP systems
- e.g. speech recognition uses a version called **word error rate (WER)**
 - the unit of operation is words instead of letters
 - divided by the number of words in gold standard
 - it can pass 100% when the system adds words

```
said = "it's hard to recognize speech".split()
```

```
heard = "it's hard to wreck a nice beach".split()
```

```
nltk.edit_distance(said, heard)
```

→ 4

```
nltk.edit_distance(said, heard) / len(said)
```

→ 0.8

HW3

- released today, due Thursday, Oct 1, 11:59 pm, on Canvas as an .ipynb file

next time

- the two classes next week will be asynchronous
 - it will be in the form of longer and more verbose slides and Python notebooks
- we'll resume in-person classes the week after on October 6